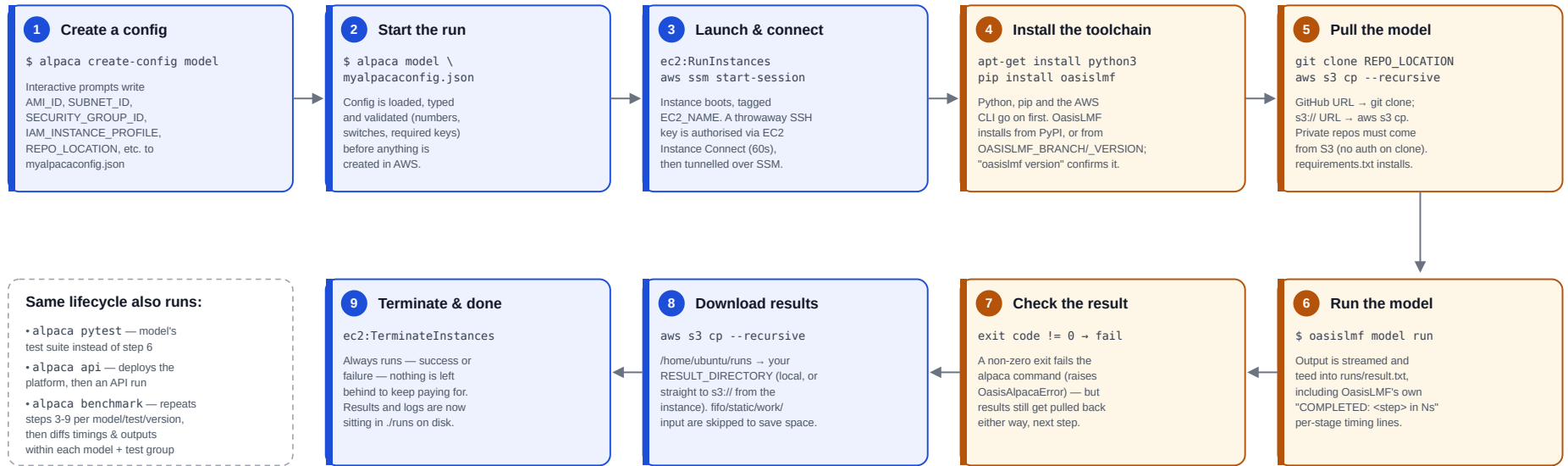


Alpaca — Model Run Workflow

What happens when you run `alpaca model <config.json>`: a throwaway EC2 instance is created, the model is pulled and run, results come home, and the instance is destroyed.

■ YOUR MACHINE

■ THROWAWAY EC2 INSTANCE



Blue = runs on your machine. Amber = runs on the throwaway EC2 instance. Setting `DEBUG=True` pauses before every command in steps 3–8 so you can step through, skip, or run something of your own.

Command reference

The exact command each step above resolves to, in order. Blue rows run on your machine; amber rows run on the EC2 instance over the SSM tunnel.

#	STEP	COMMAND	NOTES
1	Create config	<code>alpaca create-config model</code>	Also ... <code>pytest</code> , ... <code>api</code> , ... <code>benchmark</code> . Bare <code>alpaca create-config</code> prompts for the type.
2	Start the run	<code>alpaca model <config.json></code>	Validates config (required keys, numeric/switch types) before touching AWS.
3	Launch instance	<code>boto3 ec2:RunInstances</code>	Uses <code>AMI_ID</code> , <code>INSTANCE_TYPE</code> , <code>DISK_GB</code> , <code>SECURITY_GROUP_ID</code> , <code>SUBNET_ID</code> , <code>IAM_INSTANCE_PROFILE</code> .
3	Connect	<code>aws ssm start-session --document-name AWS-StartSSHSession</code>	Waits for the SSM Agent to register, retrying up to <code>SSH_MAX_RETRIES</code> (default 60 x 3s). No <code>.pem</code> key involved.
4	Install runtime	<code>apt-get install -y python3 python3-pip unzip curl</code>	Then installs AWS CLI v2 from the official zip.
4	Install OasisLMF	<code>pip install oasislmf[extra]==VERSION</code>	Or <code>git clone --branch OASISLMF_BRANCH + pip install '[extra]'</code> when a branch is set (branch wins over version). <code>oasislmf version</code> confirms the install.
5	Pull model (GitHub)	<code>git clone REPO_LOCATION</code>	Public repos only — no auth on the clone.
5	Pull model (S3)	<code>aws s3 cp --recursive s3://... /home/ubuntu</code>	Use for private models. Needs S3 read on <code>IAM_INSTANCE_PROFILE</code> .
5	Model dependencies	<code>pip install -r requirements.txt</code>	Falls back to <code>requirements.in</code> , or skips with a message if neither exists.
6	Run the model	<code>oasislmf model run</code>	Against <code>PATH_TO_OASISLMF_JSON</code> (for a benchmark <code>TESTS</code> target, that's <code>tests/<test>/oasislmf.json</code>). Output tees into <code>runs/result.txt</code> . Just before it, <code>runs/input_checksums.json</code> records SHA-256s of the <code>oasislmf.json</code> and the settings files it points to (a missing file is recorded, never fatal).
7	Result check	<code>exit code != 0 → OasisAlpacaError</code>	A failed run still fails the <code>alpaca model</code> command, but results download regardless.
8	Download results	<code>aws s3 cp --recursive ... --exclude '*/fifo/*' ...</code>	Also excludes <code>static/</code> , <code>work/</code> , most of <code>input/</code> (keeps <code>keys.csv</code> , <code>keys-errors.csv</code>). Destination is <code>RESULT_DIRECTORY</code> — local path or <code>s3://</code> bucket.
9	Terminate instance	<code>ec2:TerminateInstances</code>	Always fires, success or failure — the <code>RemoteController</code> is a context manager.

alpaca pytest

Same steps 1–5, then `pytest . -vv $PYTEST_ARGS` (plus hypothesis, mock, responses) in place of step 6. Logs land in a timestamped file under `pytest_logs/`; a failing suite fails the command.

alpaca api

Installs Docker, brings the platform up (`docker compose up -d --build` or a deploy script), polls `/healthcheck/`, then `oasislmf api run --server-url ...`. Downloads `analysis_1_output.tar.gz`.

alpaca benchmark

Runs this lifecycle once per model × test × OasisLMF version/branch (or once per model × version with `TESTS_PER_INSTANCE=shared`), then compares timings and outputs within each model + test group against a reference, writing `benchmark_report.txt`. `BENCHMARK_BUCKET` can reuse a stored baseline instead of re-running. See pages 3–5.

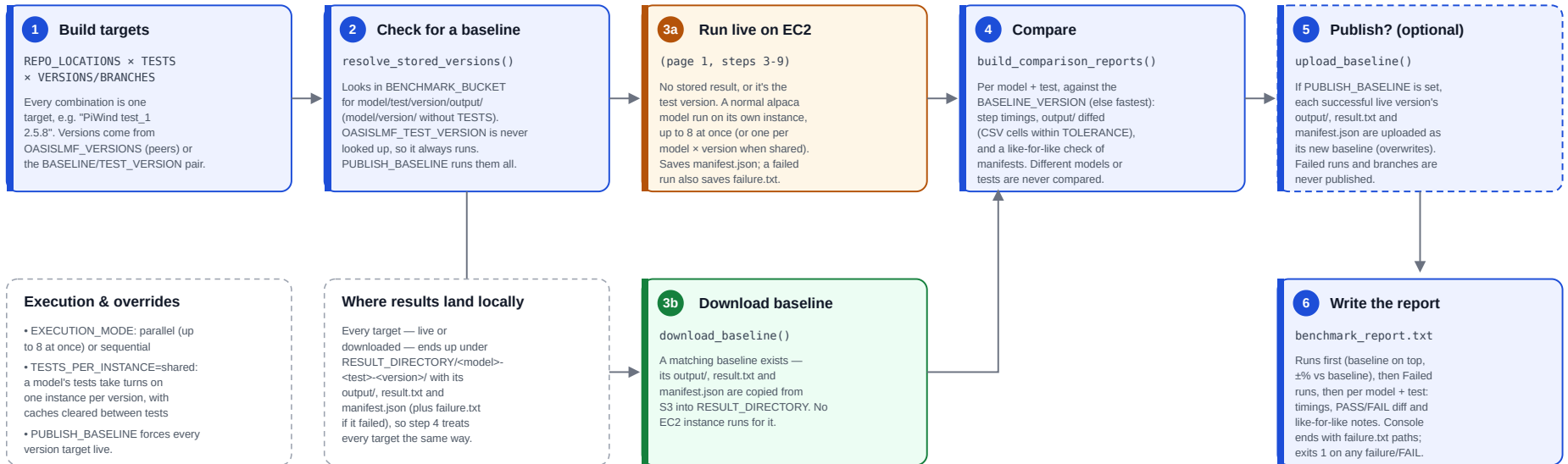
Alpaca — Benchmark Workflow

What `alpaca benchmark <config.json>` adds on top of the model-run lifecycle: one target per model × test × version, a stored S3 baseline used in place of a live run wherever one exists (except the test version, which always runs), then a timing, output and like-for-like comparison within each model + test group.

■ YOUR MACHINE

■ THROWAWAY EC2 INSTANCE

■ S3 (BENCHMARK_BUCKET) — read/written from your machine, not the instance



Blue = runs on your machine. Amber = live model run on a throwaway EC2 instance (page 1, steps 3-9). Green = S3 baseline read or write, done from your machine via boto3 — separate from the instance's own `aws s3 cp` for pulling the model or uploading RESULT_DIRECTORY.

Benchmark-specific config keys

On top of the required keys every run type shares (AMI_ID, SECURITY_GROUP_ID, SUBNET_ID, IAM_INSTANCE_PROFILE, plus exactly one of PATH_TO_OASISLMF_JSON or TESTS). Green rows are S3-only — read or written from your machine, never the instance.

KEY	DEFAULT	WHAT IT DOES
REPO_LOCATIONS	required	JSON array of model locations (GitHub URLs or s3://) — replaces the singular REPO_LOCATION used by model/pytest/api.
OASISLMF_VERSIONS / _BRANCHES	[]	One target per entry, crossed with every REPO_LOCATIONS entry. All are peers: the fastest is the reference. At least one version, branch or the pair below is required.
OASISLMF_BASELINE_VERSION / _TEST_VERSION	unset	Use instead of OASISLMF_VERSIONS (set both, different versions). The baseline is reused from BENCHMARK_BUCKET when stored and is always the reference; the test version always runs live, even if stored. Reports show each run as ±% vs baseline.
TESTS	[]	Directory names under the model's tests/ (e.g. ["test_1", "test_2"]); each runs its own tests/<test>/oasislmf.json in place of PATH_TO_OASISLMF_JSON (set one or the other, never both). Every model × test × version is its own target on its own instance, named Alpaca {model} {test} {version}. Names are case-sensitive bare directory names (no /).
TESTS_PER_INSTANCE	separate	shared runs all of a model's TESTS at one version in turn on one instance (install + model pull once; file and Numba caches cleared before each test). Results, comparison, publishing and manifests stay per test; a failing test doesn't stop the rest. Needs TESTS.
EXECUTION_MODE	parallel	parallel runs up to 8 targets at once (one EC2 instance each); sequential runs them one after another. DEBUG (stepping through each command) only works in sequential; parallel switches it off.
BENCHMARK_BUCKET	" "	s3://bucket holding baselines, keyed model/test/version/output/* and model/test/version/performance/result.txt with TESTS, or model/version/... without. The two layouts never stand in for each other. Without a bucket, every target always runs live.
PUBLISH_BASELINE	False	True forces every version target to run live (no stored baseline is reused) and uploads its result as the new baseline, overwriting any existing one. Requires BENCHMARK_BUCKET and at least one OASISLMF_VERSIONS entry.
COMPARISON_TOLERANCE	0.005	Relative tolerance for numeric CSV cells when a checksum mismatch sends the comparison to a cell-by-cell diff (OasisLMF's Monte Carlo sampling rarely produces byte-identical loss tables).
RESULT_DIRECTORY	./runs	Must be a local path for a benchmark (unlike model/api, an s3:// value is rejected) — comparison reads local files. Each target gets its own <model>-<version>/ subfolder, or <model>-<test>-<version>/ with TESTS.

Testing a new version against a published baseline

Set OASISLMF_BASELINE_VERSION (e.g. 2.5.7) and OASISLMF_TEST_VERSION (2.5.8) with BENCHMARK_BUCKET: the published baseline downloads from S3 (no EC2), the test version always runs live, and the report shows its change from the baseline, e.g. 212s, +16.3% vs baseline. Confirmed on AWS: 2.5.8 outputs identical to the stored 2.5.7 baseline.

Publishing a new baseline

Set PUBLISH_BASELINE=True with exactly the version(s) you want published. Every one of them runs live regardless of what's already stored, then step 5 uploads its output, result.txt and manifest.json, overwriting any existing baseline at that model/test/version. Failed runs are never published.

Running several tests of one model

List the test directories in TESTS, e.g. ["test_1", "test_2"] with two versions: four targets, four instances. Tests share the model's data and differ only in their own oasislmf.json (e.g. a different event set). Each test is compared only with the same test, and stored as its own model/test/version baseline. Add TESTS_PER_INSTANCE=shared to run them on one instance per version: 2 instances instead of 4.

Reading the result: report, like-for-like and failed runs

Taken from a real 2.5.7 → 2.5.8 run (model name replaced) where one test was made to fail on purpose. Both test runs shared one 2.5.8 instance; the failing test didn't stop the other.

benchmark_report.txt (excerpt)

```
Runs:
- MyModel test_does_not_exist 2.5.7 (baseline): failed (18s)
- MyModel test_does_not_exist 2.5.8: failed (21s)
- MyModel test_1 2.5.7 (baseline, from S3): success (182s)
- MyModel test_1 2.5.8: success (212s, +16.3% vs baseline)

Failed runs:
- MyModel test_does_not_exist 2.5.8: failed during model run
  The path /home/ubuntu/tests/test_does_not_exist/oasislmf.json
  (MDK config. JSON file) is indicated as preexisting but does not exist
  missing input file: tests/test_does_not_exist/oasislmf.json
  details: ./runs/MyModel-test_does_not_exist-2.5.8/failure.txt

MyModel test_1
-----
Step timings (change against the baseline, negative is faster):
oasislmf.execution.runner.run    137.55    162.75 (+18.3%)
oasislmf.manager.interface       182.34    212.13 (+16.3%)

Output comparison against MyModel test_1 2.5.7 (baseline, from S3):
PASS: Outputs identical

Like-for-like check against MyModel test_1 2.5.7 (baseline, from S3):
- MyModel test_1 2.5.8: not like for like
  - tests per instance differs: separate vs shared
```

failure.txt (in the failed run's folder)

```
Stage: model run
Error: The path /home/ubuntu/tests/test_does_not_exist/oasislmf.json
      (MDK config. JSON file) is indicated as preexisting but does not exist
Missing input files:
- tests/test_does_not_exist/oasislmf.json

Full output:
Command failed with exit status 1: ... oasislmf model run -C ...
```

End of the console output

```
Benchmark finished with failures: see the report above.
Failure details:
- MyModel test_does_not_exist 2.5.7 (baseline): ./runs/.../failure.txt
- MyModel test_does_not_exist 2.5.8: ./runs/.../failure.txt
Full report: ./runs/benchmark_report.txt
Exiting with status 1.
```

manifest.json (every run, published with baselines)

```
{ "model": "MyModel", "test": "test_1", "oasislmf_version": "2.5.8",
  "instance_type": "m6i.xlarge", "ami_id": "ami-...", "aws_region": "eu-west-1",
  "tests_per_instance": "shared",
  "inputs": { "oasislmf_json": { "path": "tests/test_1/oasislmf.json", "sha256":
    "..."},
    "model_settings_json": { ... }, "analysis_settings_json": { ... } } }
```

SITUATION	EXIT STATUS	WHAT YOU SEE
All runs succeed, outputs match	0	Runs, timings, PASS. Like-for-like notes may still appear.
A run fails	1	failure.txt in its folder (stage, key error, missing inputs, full output); "Failed runs" in the report; paths repeated at the end of the console. Other runs carry on; it's never published.
Outputs differ beyond COMPARISON_TOLERANCE	1	FAIL with the differing files in that model + test section.
Baseline failed, or fewer than two runs succeeded	1 (a run failed)	Comparison for that group skipped, with the reason. Other groups are still compared.
Runs not set up the same way	0 (reported only)	"not like for like" with what differs: instance type, AML, region, tests per instance, or a changed oasislmf.json / settings file. Baselines published before manifests existed show as unchecked.